
From World Model to Control: Evaluating Reward Structure in JEPA Latents

Naval Tapan Bhagat
University of Amsterdam
naval.tapan.bhagat@student.uva.nl

Joey Laarhoven
University of Amsterdam
joey.laarhoven@student.uva.nl

Aron Wezenberg
University of Amsterdam
aron.wezenberg@student.uva.nl

Aditya Marada
University of Amsterdam
adit.marada@student.uva.nl

Pranav Bali
University of Amsterdam
pranav.bali@student.uva.nl

Abstract

1 Joint-Embedding Predictive Architectures (JEPAs) learn latent world models
2 through predictive objectives without explicit reward supervision. However, it
3 remains unclear whether the latent representations encode information useful for
4 control. In this work, we investigate whether reward and value signals can be
5 recovered from the latent representations learned by LeWorldModel (LeWM), a
6 JEPA-based action-conditioned world model. We implement reward and value prob-
7 ing and conduct reward-aware finetuning of LeWM across multiple environments
8 to evaluate whether the latent representations can support reward-guided model
9 predictive control (MPC). Our results show that LeWM latents contain partially
10 recoverable reward-relevant structure, but performance is highly environment-
11 dependent. Reward-guided MPC succeeds in select environments, but fails to
12 consistently outperform random planning in sparse, long-horizon settings. Our find-
13 ings suggest that predictive JEPA latent spaces can encode useful reward-relevant
14 information, but recoverable reward structure alone is insufficient to guarantee
15 robust downstream control. Code is available on GitHub.

1 Introduction

17 World Models aim to learn latent representations useful for prediction and control. Classical latent
18 world models such as DreamerV3 Hafner et al. [2025] and PlaNet Hafner et al. [2019] jointly optimize
19 dynamics and reward objectives, explicitly shaping representations for downstream control. In
20 contrast, JEPA-based world models Balestriero and LeCun [2025], Maes et al. [2026] focus primarily
21 on predictive latent dynamics without explicit reward supervision. Despite lacking reward supervision,
22 JEPA-based models such as LeWorldModel (LeWM) demonstrate goal-directed latent planning,
23 suggesting that their latent spaces may encode task-relevant structure. However, it remains unclear
24 whether these predictive latent representations contain recoverable reward and value information for
25 downstream control.

26 In this work, we investigate whether reward and value signals can be recovered and learned in
27 LeWM latent representations and whether they can support downstream control. We evaluate this
28 through reward and value probing, reward-aware finetuning of LeWM, and reward-guided model
29 predictive control using learned reward models. We additionally compare LeWM representations

30 against DINOv2 Oquab et al. [2024] and DreamerV3 Hafner et al. [2025] baselines. Our contributions
31 are:

- 32 • We study reward and value probing across multiple horizons, temporal windows, and control
- 33 environments using frozen LeWM representations.
- 34 • We investigate reward-aware finetuning of LeWM while preserving predictive latent dynam-
- 35 ics through joint optimization.
- 36 • We evaluate whether learned reward models can support reward-guided model predictive
- 37 control in latent space.

38 2 Related Work

39 **World Models for Control** Latent world models such as PlaNet Hafner et al. [2019], DreamerV3
40 Hafner et al. [2025] and TD-MPC Hansen et al. [2022] jointly optimize latent dynamics together with
41 reward or value objectives, explicitly shaping representations for downstream control and planning.
42 Recent JEPA-based approaches shift focus toward predictive latent representation learning without
43 explicit reward supervision. I-JEPA Assran et al. [2023] introduced predictive joint-embedding
44 learning for visual representations, while TD-JEPA Bagatella et al. [2026] and LeWM Maes et al.
45 [2026] extend this paradigm toward reinforcement learning and world modeling. However, it remains
46 unclear whether these predictive latent representations encode reward-relevant information useful for
47 downstream control.

48 **Representation Probing** Representation probing is commonly used to evaluate whether latent
49 spaces encode task-relevant information. In contrast to prior probing work, we specifically investigate
50 whether JEPA-based world model representations contain recoverable reward and value structure
51 sufficient for downstream planning and control.

52 3 Method

53 We evaluate whether LeWM latent representations encode reward and value information useful for
54 downstream control through three components: probing, reward-aware finetuning, and reward-guided
55 planning.

56 **Reward and Value Probing** We freeze pretrained LeWM encoders and extract latent representa-
57 tions z_t from offline trajectories. Lightweight MLP and Linear probes are trained to predict either
58 immediate rewards or discounted returns from these latent states.

59 For reward probing, probes predict rewards across varying rollout horizons and temporal observation
60 windows:

$$z_{t+h} \rightarrow r_{t+h}, \quad (z_{t+i})_{i=0}^{w-1} \rightarrow r_t$$

61 For value probing, probes predict discounted Monte Carlo returns:

$$G_t = \sum_{k=0}^{T-t} \gamma^k r_{t+k}, \quad z_{t+h} \rightarrow G_{t+h}$$

62 We compare probing performance against equivalent probes trained on frozen DINOv2 Oquab et al.
63 [2024], DreamerV3 Hafner et al. [2025], and raw observation baselines.

64 **Reward-aware Finetuning** To adapt latent representations toward control objectives, we finetune
65 LeWM end-to-end with an additional reward/value prediction head while preserving the original
66 latent prediction objective. The encoder, predictor, and action encoder are jointly optimized using
67 reward/value supervision together with latent prediction and SIGReg losses.

Reward-guided Planning We evaluate whether learned reward/value models can support latent-space planning through reward-guided model predictive control (MPC). Candidate action sequences are rolled out using the LeWM predictor and scored using learned reward/value estimates:

$$a_t = \left[\arg \max_{\mathbf{a}_{t:t+H-1}} \sum_{k=0}^{H-1} \gamma^k \hat{r}_\psi(\hat{z}_{t+k}) \right]_0, \quad a_t = \left[\arg \max_{\mathbf{a}_{t:t+H-1}} \hat{G}_\psi(\hat{z}_{t+H}) \right]_0$$

Action sequences are optimized using the Cross-Entropy Method (CEM) in an MPC loop. For selected environments, we additionally compare against an oracle MPC baseline that performs CEM planning directly in the true environment using ground-truth transitions and rewards.

4 Experiments and Results

We evaluate probing, finetuning, and reward-guided planning across multiple Gymnasium Towers et al. [2025] environments spanning sparse/dense rewards, and discrete/continuous action spaces (Table 1). Additional training details and hyperparameters are provided in Appendix A.

Table 1: Gymnasium Environments for running probing, finetuning, and planning experiments. *CartPoleExplore-v0 is a custom environment. Details in Appendix B.

Environment	Reward Space	Action Space
ALE/Pong-v5	Sparse	Discrete
LunarLander-v3	Dense	Discrete
CarRacing-v3	Dense	Continuous
Hopper-v5	Dense	Continuous
FetchPickAndPlace-v4	Sparse	Continuous
FetchPickAndPlaceDense-v4	Dense	Continuous
MountainCar-v0	Sparse	Discrete
CartPoleExplore-v0*	Dense	Discrete

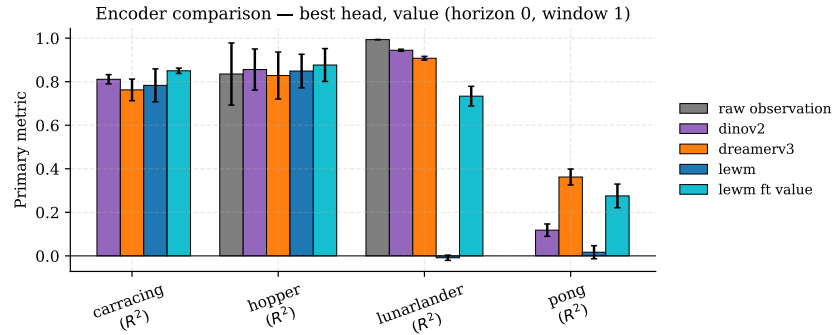


Figure 1: Value probing performance compared against DINOv2 and DreamerV3 encoder baselines. Bars show the mean and standard deviation.

Reward and Value Probing LeWM latent representations contain partially recoverable reward and value structure, although performance is highly environment dependent (Figure 1). In several environments, LeWM probes perform competitively with DreamerV3 and DINOv2 baselines, while in others performance collapses toward mean prediction behavior.

Increasing rollout horizons generally degrades probing quality due to accumulated latent rollout errors (Figures 2). In contrast increasing temporal observation windows can improve performance in environments with strong motion dynamics such as Pong and MountainCar, suggesting that stacked observations help infer information like velocity and direction (See Appendix C and E for more details).

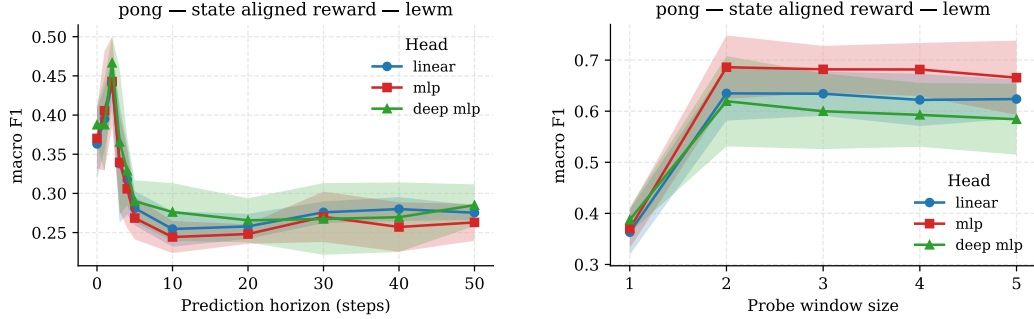


Figure 2: Pong reward probing. Left: performance vs. predictor horizon h . Right: performance vs. probe window size w . Shaded bands show seed variation as std.

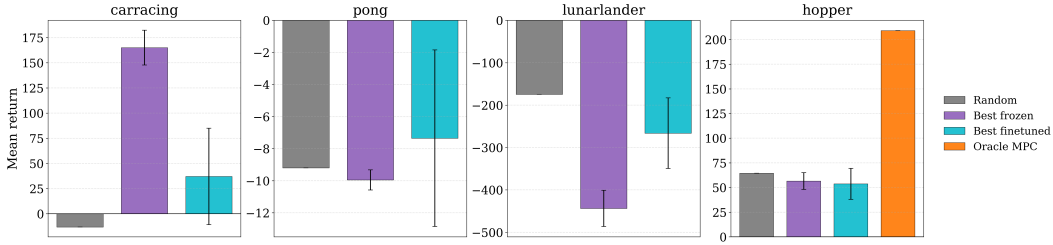


Figure 3: Reward-guided MPC on representative environments. Bars show mean return; error bars show standard deviation across seeds. Oracle MPC uses true simulator rewards.

Reward-aware Finetuning We evaluate whether explicit reward supervision can adapt LeWM latent representations toward reward prediction while preserving the pretrained latent dynamics. Joint optimization of the reward, prediction, and SIGReg losses stabilizes finetuning across all 8 environments without representation collapse: all three losses decrease together (Appendix Fig. 14), and test R^2 improves over frozen-probe baselines on 11 of 15 (env, target) cells (Appendix Fig. 15), indicating that reward supervision is broadly compatible with the LeWM objective.

Reward-guided MPC Reward-guided MPC performance depends heavily on the quality and stability of latent rollouts (Figure 3). While value-guided planning succeeds in select environments, particularly CarRacing, most environments fail to consistently outperform random planning baselines, particularly in sparse and long-horizon settings such as Pong and LunarLander. In Hopper, oracle reward-guided MPC substantially outperforms latent reward-guided MPC, indicating that errors in learned latent dynamics and reward estimation significantly degrade planning quality.

These results suggest that recoverable reward structure alone is insufficient to guarantee reliable downstream control. Errors accumulated during rollout prediction can significantly degrade planning quality even when offline reward probing performs well.

5 Conclusion

We investigated whether predictive latent representations learned by JEPA-based world models encode reward and value information useful for downstream control. Using LeWM, we evaluated this question through probing, reward-aware finetuning, and reward-guided MPC across multiple control environments.

Our experiments show that LeWM latent spaces contain partially recoverable reward-relevant structure, although probing performance remains highly environment-dependent. Temporal observation windows improved performance in environments with strong motion dynamics, while increasing rollout horizons generally degrades prediction quality due to accumulated latent rollout errors. Reward-aware finetuning further suggests that reward supervision can be incorporated without imme-

112 diately destroying the predictive latent structure when jointly optimized with the latent prediction
113 objectives.

114 However, reward-guided planning reveals a significant gap between reward recoverability and down-
115 stream controllability. While latent MPC succeeds in selected environments, comparisons against
116 the oracle MPC demonstrate that learned latent rollouts remain substantially weaker than planning
117 directly in the true environment. These results suggest that recoverable reward structure alone is
118 insufficient to guarantee reliable latent-space control and that stable long-horizon dynamics remain a
119 major challenge for JEPA-based world models.

120 Future work may investigate hybrid predictive-control objectives, latent consistency regularization,
121 and planning-aware representation learning to improve controllable world models.

References

- Mahmoud Assran, Quentin Duval, Ishan Misra, Piotr Bojanowski, Pascal Vincent, Michael Rabbat, Yann LeCun, and Nicolas Ballas. Self-supervised learning from images with a joint-embedding predictive architecture. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 15619–15629, 2023. URL https://openaccess.thecvf.com/content/CVPR2023/html/Assran_Self-Supervised_Learning_From_Images_With_a_Joint-Embedding_Predictive_Architecture_CVPR_2023_paper.html.
- Marco Bagatella, Matteo Pirota, Ahmed Touati, Alessandro Lazaric, and Andrea Tirinzoni. Td-jepa: Latent-predictive representations for zero-shot reinforcement learning. In *The Fourteenth International Conference on Learning Representations*, 2026. URL <https://openreview.net/forum?id=SzXDuBN8M1>. Oral.
- Randall Balestriero and Yann LeCun. Lejpa: Provable and scalable self-supervised learning without the heuristics, 2025. URL <https://arxiv.org/abs/2511.08544>.
- Danijar Hafner, Timothy Lillicrap, Ian Fischer, Ruben Villegas, David Ha, Honglak Lee, and James Davidson. Learning latent dynamics for planning from pixels. In *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pages 2555–2565. PMLR, 2019. URL <https://proceedings.mlr.press/v97/hafner19a.html>.
- Danijar Hafner, Jurgis Pasukonis, Jimmy Ba, and Timothy Lillicrap. Mastering diverse control tasks through world models. *Nature*, 640(8059):647–653, 2025. doi: 10.1038/s41586-025-08744-2. URL <https://www.nature.com/articles/s41586-025-08744-2>.
- Nicklas Hansen, Hao Su, and Xiaolong Wang. Temporal difference learning for model predictive control. In *Proceedings of the 39th International Conference on Machine Learning*, volume 162 of *Proceedings of Machine Learning Research*, pages 8387–8406. PMLR, 2022. URL <https://proceedings.mlr.press/v162/hansen22a.html>.
- Lucas Maes, Quentin Le Lidec, Damien Scieur, Yann LeCun, and Randall Balestriero. Leworldmodel: Stable end-to-end joint-embedding predictive architecture from pixels, 2026. URL <https://arxiv.org/abs/2603.19312>.
- Maxime Oquab, Timothée Darcet, Théo Moutakanni, Huy V. Vo, Marc Szafraniec, Vasil Khali-dov, Pierre Fernandez, Daniel Haziza, Francisco Massa, Alaaeldin El-Nouby, Mahmoud Assran, Nicolas Ballas, Wojciech Galuba, Russell Howes, Po-Yao Huang, Shang-Wen Li, Ishan Misra, Michael Rabbat, Vasu Sharma, Gabriel Synnaeve, Hu Xu, Hervé Jegou, Julien Mairal, Patrick Labatut, Armand Joulin, and Piotr Bojanowski. DINOv2: Learning robust visual features without supervision. *Transactions on Machine Learning Research*, 2024. ISSN 2835-8856. URL <https://openreview.net/forum?id=a68SUt6zFt>.
- Mark Towers, Ariel Kwiatkowski, Jordan Terry, John U. Balis, Gianluca De Cola, Tristan Deleu, Manuel Goulão, Andreas Kallinteris, Markus Krimmel, Arjun KG, Rodrigo Perez-Vicente, Andrea Pierré, Sander Schulhoff, Jun Jet Tai, Hannah Tan, and Omar G. Younis. Gymnasium: A standard interface for reinforcement learning environments. In *Advances in Neural Information Processing Systems 38, Datasets and Benchmarks Track*, 2025. URL <https://openreview.net/forum?id=qPMLvJxtPK>. Spotlight.

A Experimental Setup

A.1 Datasets and Preprocessing

Dataset collection Offline datasets were generated separately for each environment using custom trajectory collection scripts. Depending on the environment, datasets were collected using heuristic policies, PPO-trained policies, expert SB3 baseline policies, or a mixture of expert policies and random stochastic policies. This ensured sufficient trajectory diversity and reward coverage.

Collected trajectories store, in general, image observations, actions, rewards, done signals, and auxiliary metadata required for probing, finetuning, or planning. Collection scripts and corresponding configurations are released in the codebase.

Dataset preprocessing All environments use image-based observations collected from offline trajectories. To reduce repeated image decoding and preprocessing overhead during training, observations were preprocessed offline through resizing and caching into uncompressed tensor formats prior to training. This preprocessing significantly reduced LeWM training time and dataloader overhead, reducing environment-dependent training times to approximately 1–5 hours depending on dataset size and environment complexity.

Image observations were resized to 224x224 during data collection, prior to training.

A.2 LeWM and DreamerV3 training

LeWM and DreamerV3 (small) models were trained separately for each environment, using the generated offline datasets where applicable. Pretrained checkpoints from completed training runs were used for further probing, finetuning and planning experiments.

LeWM training follows the original action-conditioned latent prediction setup together with SIGReg. DreamerV3 (small) models were training using the official DreamerV3 implementation.

Training configurations and scripts will be released in the codebase.

A.3 Reward/Value Probing

Reward and Value probing evaluates whether frozen latent representations encode recoverable reward structure.

Probe Inputs

Probes are trained on (latent) representations extracted from:

- pretrained LeWM encoders
- DreamerV3 (small) encoders
- DINOv2 encoder
- Raw state observations.

For LeWM and DreamerV3, probes are evaluated across multiple encoder seeds. Unless otherwise specified, encoder seeds $\{7, 42, 314\}$ are used. DINOv2 uses a single pretrained checkpoint ($s = 11$). Raw state observations are only evaluated on environments where a low-dimensional state vector is available (CartPoleExplore, MountainCar, Hopper, LunarLander, FetchPickAndPlace), excluding image-only environments such as Pong and CarRacing.

Probe Architectures

Both linear and lightweight MLP probes are evaluated:

- **Linear**: a single linear layer.
- **MLP**: a two-layer MLP with ReLU activations and hidden size 256.
- **Deep MLP**: a three-layer MLP with ReLU activations and hidden size 256.

Two prediction targets are evaluated per environment:

- **Reward**: the state-aligned immediate reward r_t at the selected horizon. For CartPoleExplore and FetchPickAndPlace (sparse), this is a binary classification target; for Pong it is a three-class classification target ($\{-1, 0, +1\}$); for all other environments it is a regression target.
- **Value / Discounted return**: the Monte Carlo return-to-go $G_t = \sum_{k=0}^{T-t} \gamma^k r_{t+k}$ with $\gamma = 0.99$, always treated as a regression target.

For binary and Pong reward targets, class-weighted loss is applied to address class imbalance. For binary targets, the positive-class weight is set to $w_+ = N_- / N_+$, where N_- and N_+ are the number of negative and positive training examples respectively. For the multiclass Pong target, per-class weights are computed analogously using inverse class frequencies.

215 Horizon and Temporal Window Sweeps

216 Probe training is performed across prediction horizons $h \in \{0, 1, 2, 3, 4, 5, 10, 20, 30, 40, 50\}$
217 (LeWM only; all other encoders use $h = 0$). Temporal window sizes $w \in \{1, 2, 3, 4, 5\}$ are
218 swept at $h = 0$; horizon sweeps use $w = 1$.

219 Probing Datasets

220 For each environment, we extract latent representations from the same trajectory dataset used to
221 pretrain the corresponding LeWM. Observations are encoded in batches of 64 using each frozen
222 encoder. For LeWM, latents are extracted at horizon offsets $h \in \{0, 1, \dots, 50\}$ by rolling out the
223 predictor from the encoded observation. For DreamerV3 and DINOv2, only $h = 0$ is used since these
224 encoders do not support offline latent rollouts. DINOv2 uses the `dinov2_vits14` variant.

225 Training Details

226 The trajectory dataset is split at the episode level using fixed 80/10/10 ratios for train, validation,
227 and test respectively. Splitting by episode avoids temporal leakage between splits. Early stopping is
228 monitored on the validation loss; all reported figures and tables use the held-out test split.

229 All probes are trained with Adam ($\text{lr} = 10^{-3}$), a batch size of 256, for up to 75 epochs with early
230 stopping (patience 5, monitored on the validation loss). Three probe seeds are used ($\{7, 23, 42\}$),
231 crossed with all encoder seeds, yielding at least 9 runs per configuration for LeWM and DreamerV3.

232 The primary metric is chosen per objective type: R^2 for regression targets, F1 for binary classification,
233 and macro-F1 for Pong’s three-class reward target. All reported values are computed on the held-out
234 test split. Figures show mean $\pm$ standard deviation across probe seeds and encoder seeds.

235 A.4 Reward-aware Finetuning

236 Reward-aware Finetuning attaches an additional reward/value prediction head to the pretrained
237 LeWM. It jointly optimizes:

- 238 • reward/value prediction loss
- 239 • latent prediction loss
- 240 • SIGReg loss

241 The encoder, predictor, and action-encoder are jointly optimized during finetuning to preserve
242 predictive latent dynamics while adapting representations toward downstream control objectives. For
243 horizon-0 finetuning, reward supervision directly updates the encoder, projector, and reward/value
244 head, while the predictor and action-encoder components continue receiving gradients only through
245 the latent prediction objective. For horizons $h > 0$, reward/value supervision additionally updates the
246 action encoder and predictor.

247 Finetuning experiments use Adam optimization with per-cell tuned learning rates in the range
248 2.5×10^{-5} to 8.6×10^{-5} (median $\approx 6.5 \times 10^{-5}$). Batch size is 256, reduced to 128 for the largest
249 datasets (e.g., Pong). Models train for 5–10 epochs, taking approximately 3–30 minutes per run on a
250 single H100 GPU depending on dataset size and rollout horizon. Full configurations are available in
251 the codebase.

252 Per-cell hyperparameters are selected via an Optuna sweep (TPE sampler) over `lr`, the three loss
253 weights, and `max_epochs`, with validation loss as the objective; per-cell trial counts range from 15 to
254 147 (median ~ 40). Recipes are selected by a JEPA-preserving rule: the maximum validation R^2 is
255 subject to no increase in validation predictor loss over the pretrained baseline ($\Delta \text{pred_loss}_{\text{val}} \leq 0$),
256 satisfied by at least one trial in all 15 cells. The seed-42 recipe is then re-applied unchanged to LeWM
257 seeds 7 and 314 for a 3-seed transfer, and test metrics are aggregated as mean $\pm$ std across these three
258 seeds on a held-out 10% episode-stratified split never seen during pretraining or finetuning.

259 A.5 Reward/Value-guided MPC

260 Reward-guided Model Predictive Control evaluates whether learned reward models can support
261 latent-space planning.

Latent planning Candidate action sequences are rolled out using the LeWM latent predictor and scored using either learned reward estimates or learned value estimates.

Cross-Entropy Method Planning uses CEM for iterative trajectory optimization.

For discrete-action environments, categorical CEM sampling is used. For continuous control environments, Gaussian CEM sampling is used with iterative mean and variance updates over elite action sequences.

Unless otherwise specified, MPC uses:

- planning horizon $H = 5$
- 5 CEM optimization iterations
- 128 candidate trajectories per iteration
- 16 elite trajectories for the CEM update

For LunarLander, we used $H = 8$, 256 candidates, and 32 elites. For the oracle simulator-MPC diagnostic, we used 256 candidates and 32 elites due to the smaller number of evaluated environments.

For the MountainCar $h = 10$ return-probe condition, the planning horizon was set to $H = 10$.

Oracle-Simulator MPC For selected environments, we additionally evaluate oracle-simulator MPC baselines using direct environment rollouts and ground truth rewards instead of latent imagination. This provides an approximate upper bound on planning performance using the same CEM optimization procedure.

Evaluation details Unless otherwise specified, MPC evaluations use fixed planning seeds ($s = 17$) across all experiments to reduce variance between planning runs. The reported values are averages based on probes trained on the 3-seed pretrained LeWMs.

Depending on environment complexity and rollout source (latent imagination vs oracle environment rollouts), MPC evaluations require approximately 5 minutes to 1 hour per experiment.

All methods were evaluated from dataset-sampled initial states using a fixed environment-specific episode budget; we report mean returns over 20–50 evaluation episodes depending on environment (Table 2).

Environment	Evaluation episodes	Evaluation budget
CartPole	50	100
MountainCar	50	200
Hopper	50	50
CarRacing	20	300
Pong	20	500
LunarLander	30	300
FetchPickAndPlace	30	50
FetchPickAndPlaceDense	30	50

Table 2: Evaluation settings for reward-guided MPC. The evaluation budget denotes the maximum number of real environment steps per episode.

Due to computational cost, the oracle simulator-MPC diagnostic was evaluated on 20 episodes for Hopper and FetchPickAndPlaceDense, using the same episode budgets as the main MPC runs.

A.6 Compute Resources

All experiments were conducted primarily on NVIDIA-A100 (40GB) GPUs provided through the Snellius supercomputing infrastructure.

Probe training runs required 10–30 minutes per experiment depending on environment and probe architecture. Reward-aware finetuning required approximately 10 minutes to 2 hours per environment.

295 LeWM pretraining required between 1–5 hours depending on dataset size and environment complexity.
296 MPC evaluations varied substantially depending on environment complexity and whether planning
297 used latent imagination or oracle environment rollouts.

298 **B Environment details**

299 **B.1 CartPoleExplore-v0**

300 CartPoleExplore-v0 is a modified version of CartPole-v1 to encourage broader state-space coverage.
301 The following changes were introduced:

- 302 1. The initial cart position is sampled uniformly from $[-2.0, 2.0]$.
- 303 2. The initial pole orientation is sampled uniformly from $[-\pi, \pi]$.
- 304 3. The initial cart velocity and pole angular velocity are sampled uniformly from $[-1.0, 1.0]$.
- 305 4. The reward is binary: the agent receives reward 1 when the cart position remains within
306 $[-2.4, 2.4]$ and the pole angle remains within $\pm 12^\circ$ and 0 otherwise.
- 307 5. Pole angle is no longer used as a termination condition. An episode terminates only when
308 the cart position leaves the range $[-2.4, 2.4]$, corresponding to the cart moving outside the
309 visible frame.

310 **C Additional Results**

311 We ran various probing ablations, including multiple horizons $h \in \{0, 1, 2, 3, 4, 5, 10, 20, 30, 40, 50\}$
312 and window sizes $w \in \{1, 2, 3, 4, 5\}$, for both reward and discounted return-to-go targets. We also
313 ran action-conditioned and non-action-conditioned variants for the $h = 0$ and $w = 1$ settings. Below
314 are some of the interesting results from these ablations.

315 **C.1 Value and reward probing**

316 Figures 4–7 summarise probing performance for discounted return and reward, across the subset of
317 environments analysed in the main text and the remaining environments. They serve as a reference
318 for the ablations in the following subsections.

319 **C.2 Short predictor rollouts improve reward and return retrievability**

320 In several environments, a short predictor rollout ($h > 0$) makes reward or return information easier
321 to recover from the latent than the encoded latent alone ($h = 0$). Figure 8 shows this for MountainCar
322 discounted return and Pong reward. This pattern is consistent with the predictor injecting transition-
323 relevant information, conditioned on the previous latents and actions, that is not directly present in
324 the single-step encoded latent. For example, this information could include velocity, which is highly
325 relevant for both Pong and MountainCar.

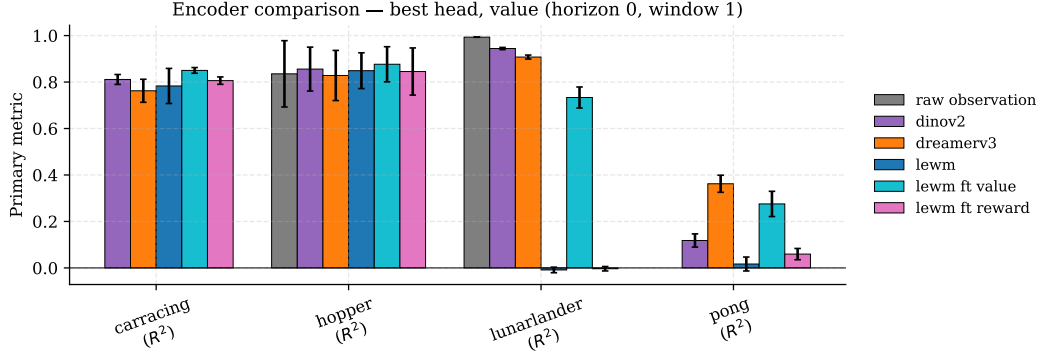


Figure 4: Discounted return probing performance on the subset of environments analysed in the main text, compared against DINOv2 and DreamerV3 encoder baselines. Bars show the mean and standard deviation.

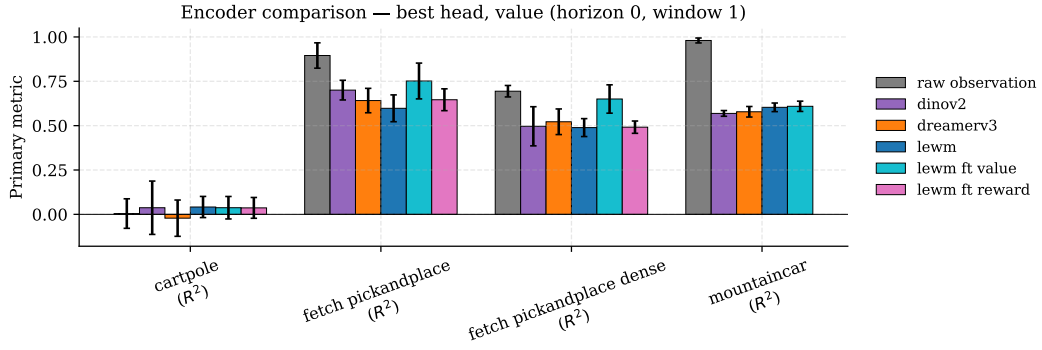


Figure 5: Discounted return probing performance on the remaining environments, compared against DINOv2 and DreamerV3 encoder baselines. Bars show the mean and standard deviation.

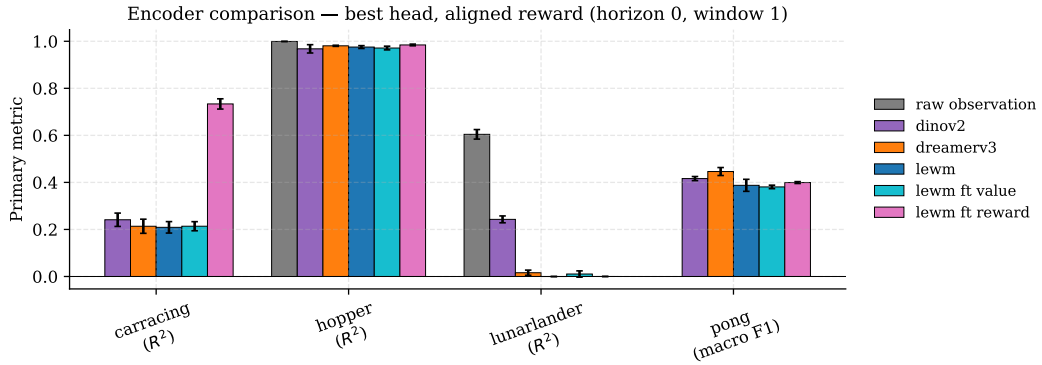


Figure 6: Reward probing performance on the subset of environments analysed in the main text, compared against DINOv2 and DreamerV3 encoder baselines. Bars show the mean and standard deviation.

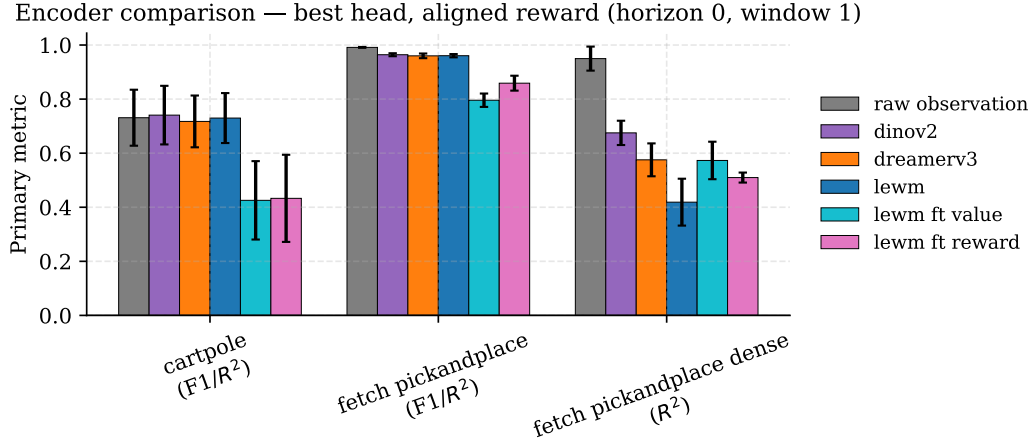


Figure 7: Reward probing performance on the remaining environments, compared against DINOv2 and DreamerV3 encoder baselines. Bars show the mean and standard deviation.

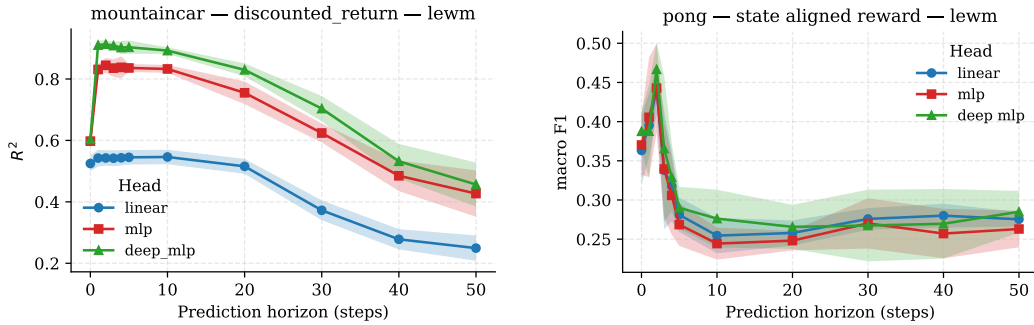


Figure 8: Probing performance over predictor horizon h for MountainCar (discounted return, left) and Pong (reward, right). Short predictor rollouts improve probing relative to $h = 0$, with the effect appearing at $h = 1$ in MountainCar and around $h = 2$ in Pong. Graphs show the mean and standard deviation.

326 C.3 Temporal windows produce a similar effect

327 Explicit temporal windows produce a qualitatively similar improvement to short predictor rollouts.
 328 Figure 9 shows that increasing the probe window size from $w = 1$ improves both MountainCar return
 329 probing and Pong reward probing. This supports the interpretation that the predictor and the temporal
 330 window both expose motion- or transition-related information that is difficult to recover from a single
 331 encoded latent.

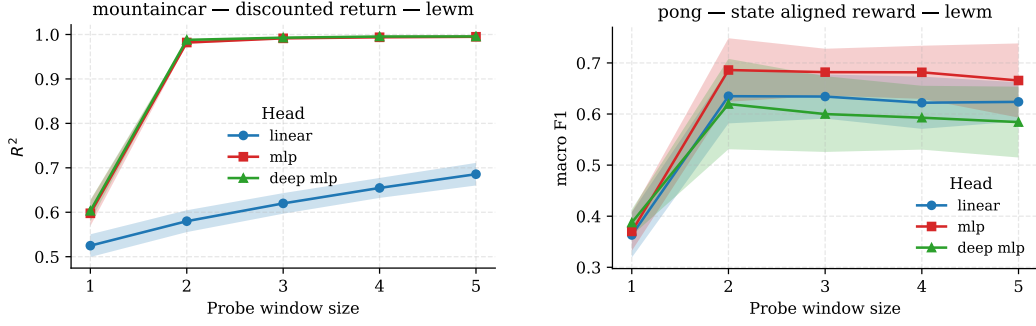


Figure 9: Window-size sweeps for MountainCar discounted return (left) and Pong reward (right). Increasing w from 1 improves probing in both environments, mirroring the effect of short predictor rollouts in Figure 8. Graphs show the mean and standard deviation.

332 C.4 The horizon effect requires a minimum level of reward retrievability

333 The improvement from short predictor rollouts is not universal. Figure 10 compares the CarRacing
 334 reward horizon sweep before and after reward finetuning. The non-finetuned model does not show a
 335 short-horizon improvement, while the reward-finetuned model does. This suggests that the predictor
 336 can only amplify reward information that is already present in the latent dynamics to some degree:
 337 when the base model encodes very little reward-relevant information, the predictor has nothing to
 338 expose.

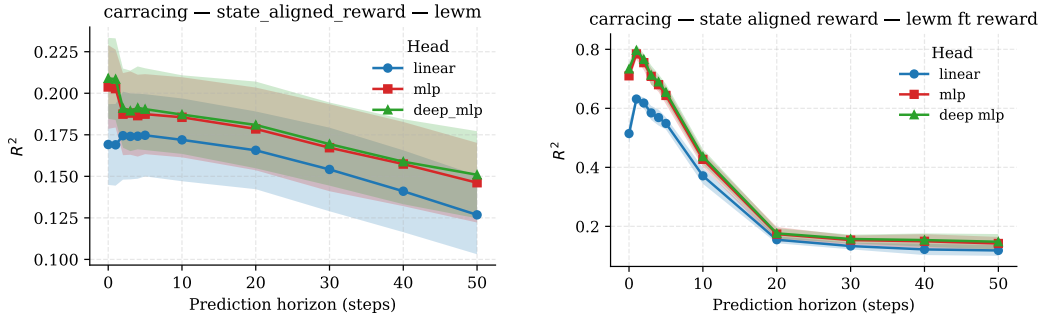


Figure 10: CarRacing reward probing over predictor horizon for the non-finetuned LeWM (left) and the reward-finetuned LeWM (right). Only the reward-finetuned model shows a short-horizon improvement, suggesting a minimum level of reward retrievability is required before the predictor can help. Graphs show the mean and standard deviation.

339 C.5 Long predictor horizons degrade probe performance

340 Across environments, long predictor rollouts reduce probing quality. Figure 11 shows this for Hopper
 341 reward and CarRacing discounted return; the same trend is visible in the MountainCar and Pong
 342 sweeps in Figure 8. The decay rate varies by environment: some degrade quickly, others more
 343 gradually. Most curves plateau at a non-trivial value rather than collapsing to zero, likely because the
 344 probe can fall back on a mean-prediction or near-terminal-value heuristic that retains some predictive
 345 power without true state information.

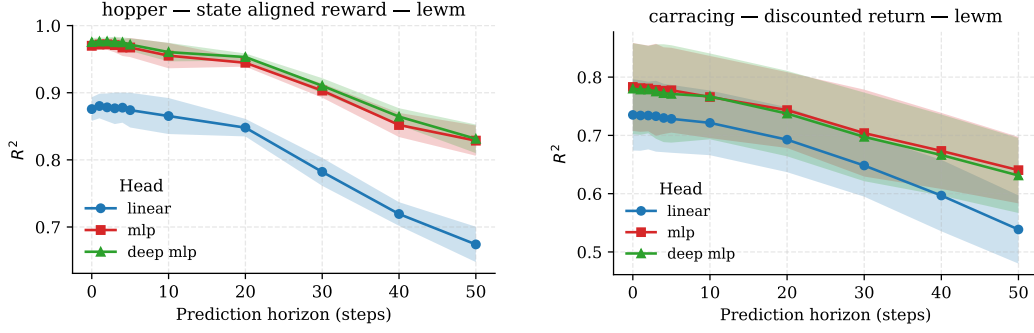


Figure 11: Long-horizon decay of probing performance for Hopper reward (left) and CarRacing discounted return (right). The decay rate varies across environments, but most curves plateau above zero rather than collapsing. Graphs show the mean and standard deviation.

346 C.6 Action conditioning is required for action-dependent reward

347 In environments where reward depends explicitly on the action, action information appears to be
 348 needed for stronger reward or return probing. Figure 12 compares LunarLander reward probing with
 349 and without action conditioning across all four encoders (LeWM, DINOv2, DreamerV3, and the
 350 raw-observation baseline) and across probe head types. Adding action information yields a clear
 351 improvement in every encoder/head combination, confirming the importance of action conditioning
 352 when the reward depends on the action. In LunarLander, engine firings directly contribute to the
 353 reward signal, so a probe that does not observe the action cannot reconstruct that component from
 354 latents alone.

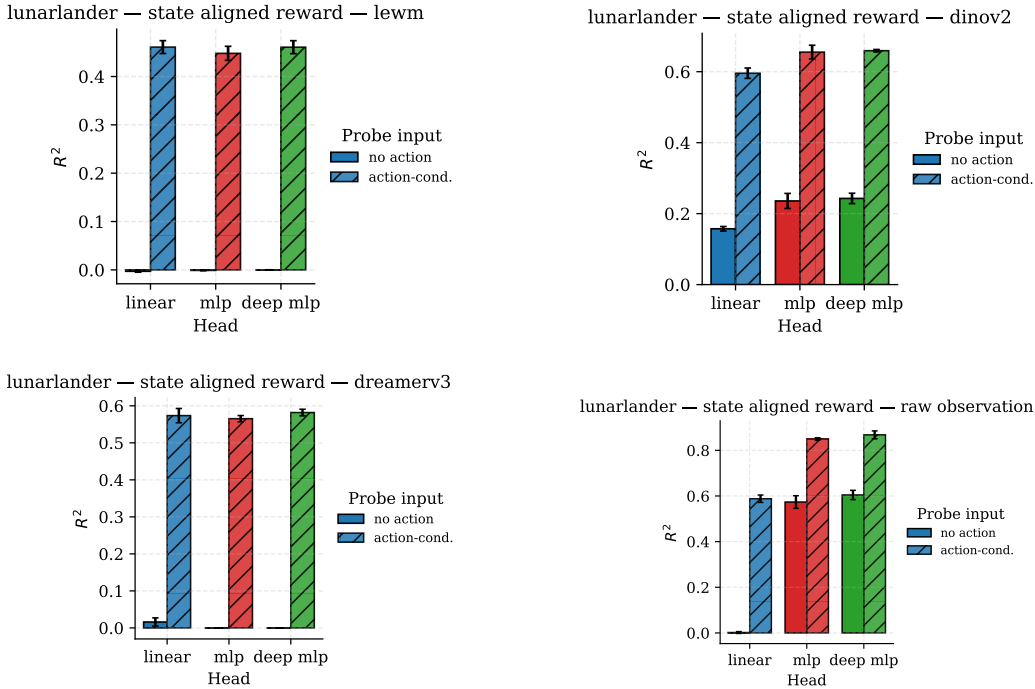


Figure 12: LunarLander reward probing with and without action conditioning, per encoder and per probe-head type. Top: LeWM (left), DINOv2 (right). Bottom: DreamerV3 (left), raw observation (right). Action conditioning improves probing across every encoder and head, indicating that action information is important whenever the reward depends on the action. Bars show the mean and standard deviation.

355 C.7 Additional MPC results

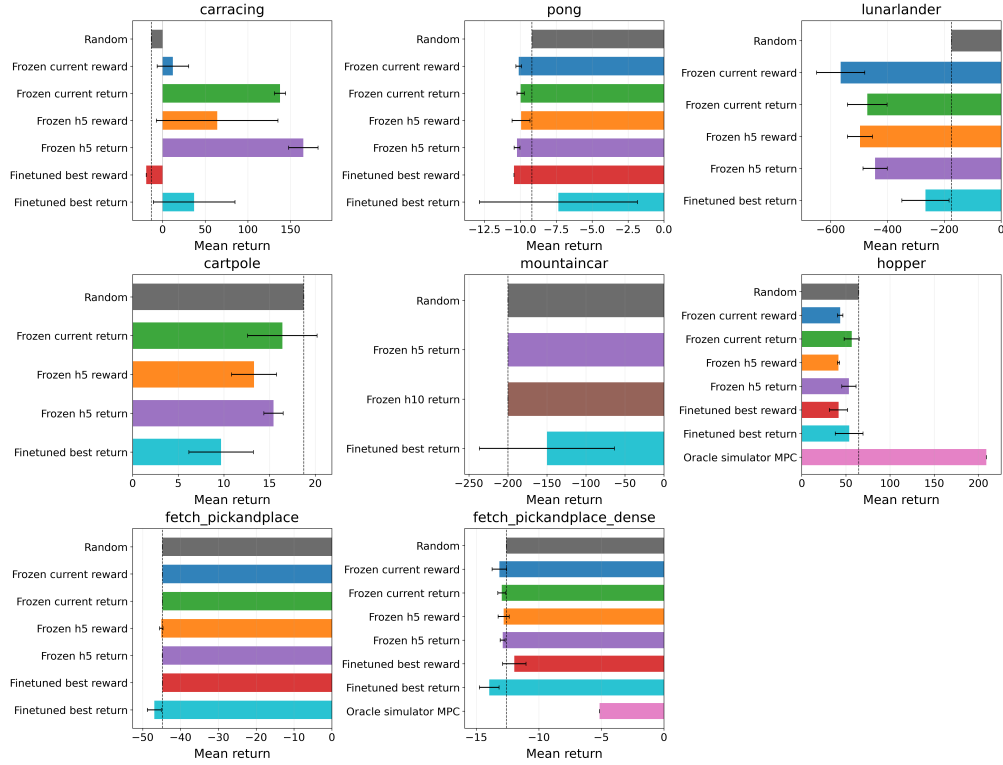


Figure 13: Full MPC ablation across probe types. Bars show mean return over evaluation episodes; error bars show seed standard deviation. Dashed line marks the random baseline.

356 C.8 Additional Finetuning results

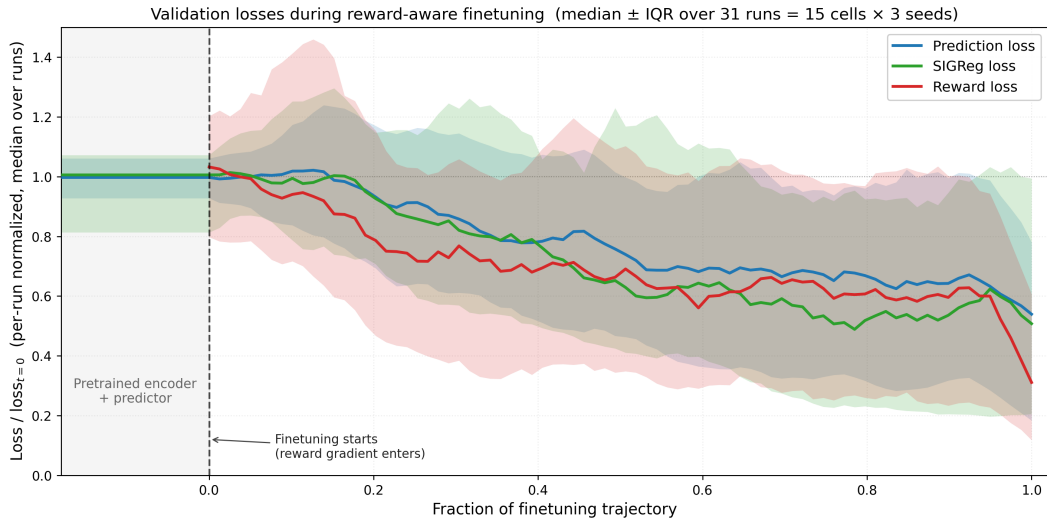


Figure 14: Validation losses during reward-aware finetuning, normalized by each run's first-5-step median (so losses are proportionate) and aggregated as median $\pm$ IQR over 31 runs. All three losses decrease together, showing that the reward objective is compatible with the SIGReg and JEPA-preservation objectives.

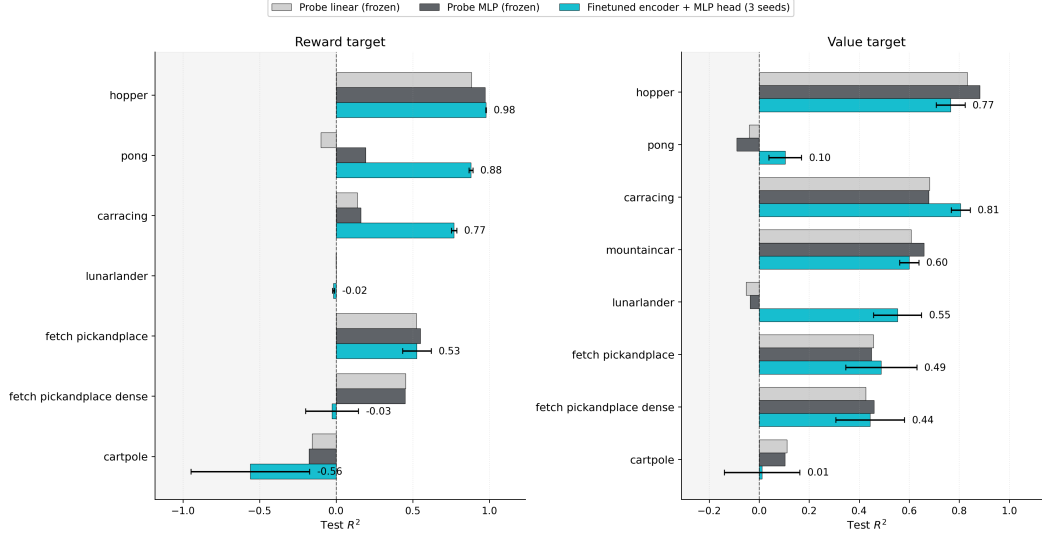


Figure 15: Held-out test R^2 for reward (left) and value (right) targets across all 8 envs: frozen linear probe, frozen MLP probe, finetuned encoder + MLP head (3 seeds, mean $\pm$ std). Mountaincar has no reward target.

357 D Dreamer-Like Actor-Critic Exploration

358 We also test whether Dreamer-style actor-critic refinement on top of a frozen LeWM can recover
 359 competent control from imagined rollouts. The canonical recipe (single-frame latent, MSE critic,
 360 λ -returns with $\gamma=0.99$, $\lambda=0.95$, learned continue head) fails across all three environments and three
 361 encoder seeds, with imagined returns drifting far above real-environment returns. We diagnose three
 362 failure modes and propose AC-v2, which restores learning on MountainCar.

363 **Setup** The frozen backbone provides an encoder $\phi : \mathbb{R}^{224 \times 224 \times 3} \rightarrow \mathbb{R}^{192}$ and a latent predictor
 364 $\pi_{\text{LeWM}} : (z_t, a_t) \mapsto \hat{z}_{t+1}$. We attach two frozen probes (reward $\hat{r}(z)$, continuation $\hat{c}(z)$; Appendix A)
 365 and learn a stochastic actor π_θ (categorical or tanh-Normal) and a value V_ψ , both width-256 MLPs.
 366 Only θ, ψ are updated; the actor never queries the real environment.

367 **Algorithm** Each update samples 128 start latents from a precomputed feature cache and rolls
 368 predictor+actor for H_{imag} steps. λ -returns are

$$R_t^\lambda = \hat{r}_t + \gamma \hat{c}_t [(1-\lambda) V_\psi(\hat{z}_{t+1}) + \lambda R_{t+1}^\lambda], \quad R_{H_{\text{imag}}}^\lambda = V_\psi(\hat{z}_{H_{\text{imag}}}),$$

369 with $\gamma=0.99$, $\lambda=0.95$. The actor maximizes $\mathbb{E}_t[\log \pi_\theta(a_t | z_t)(R_t^\lambda - V_\psi(z_t))]$ with entropy bonus
 370 10^{-2} ; the critic regresses MSE onto $\text{sg}(R_t^\lambda)$. Adam, lr 3×10^{-4} , $H_{\text{imag}}=15$. This is the AC-v1
 371 baseline.

372 **AC-v1 sweep** We swept $\{\text{CartPole}, \text{MountainCar}, \text{Hopper}\} \times \text{seeds } \{7, 42, 314\} \times \text{two probe vari-}$
 373 $\text{ants } (h_0: \text{immediate-reward; rollout10: } h=10 \text{ rolled-out latents})$. All 18 runs collapsed to environment
 374 floors (Table 3). Per-update logs reveal three consistent symptoms:

- 375 • **Model exploitation.** The actor drives imagined trajectories off the encoder manifold, where
 376 the probes (trained only on encoded real frames) are unconstrained. Imagined-vs-real gaps
 377 are large across the board (Figure 16): Hopper 62–125 vs -0.4 – 4.8 ; CartPole 80–102 vs
 378 1–3; MountainCar rollout10 164–290 vs exactly -200 . Rollout-trained probes amplify
 379 rather than mitigate the gap.
- 380 • **Continuation collapse.** The continue head saturates at ≈ 0.99 throughout imagination, so
 381 episodes effectively never terminate and discounting degenerates into an undiscounted sum
 382 of probe outputs along a drifting trajectory.

- **Entropy collapse.** Actor entropy falls 2–3 orders of magnitude within a run (e.g. CartPole 0.66 $\rightarrow$ 0.04 by 10k updates, Hopper 2.76 $\rightarrow$ 0.18 by 35k). The policy commits early to a deterministic exploit of the imagined-return gap.

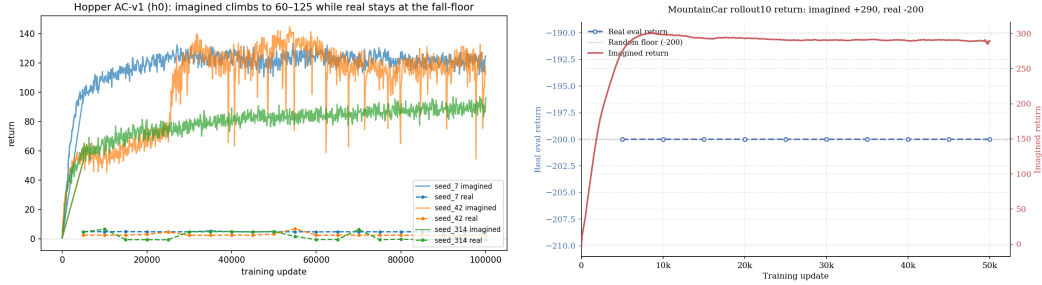


Figure 16: AC-v1 imagined-vs-real return gap. **Left:** Hopper with the h_0 reward probe across three encoder seeds; solid imagined returns reach 80–125 while dashed real-env evals stay ≤ 5 . **Right:** MountainCar with the rollout10 return probe (seed 42); imagined return (solid, right axis) climbs to $\approx +290$ while real return (dashed, left axis) is pinned at -200 . The gap is the model-exploitation signature.

Table 3: Real-environment final eval return for the naive single-frame AC-v1 baseline. All 18 runs collapse to environment floors, ≤ 5 for Hopper (random fall), ≤ 3 for CartPole, exactly -200 for MountainCar, while imagined returns climb to 80–290. Higher is better.

Seed	CartPole		MountainCar		Hopper	
	h_0	rollout10	h_0	rollout10	h_0	rollout10
7	3.0	1.0	-200.0	-200.0	4.84	0.82
42	1.0	1.0	-200.0	-200.0	2.65	4.83
314	2.0	1.0	-200.0	-200.0	-0.40	0.02

Diagnosis Targeted MountainCar diagnostics localize the failure. (i) *State observability*: ground-truth velocity is barely recoverable from a single LeWM frame ($R^2=0.42$) but well-decoded from two consecutive frames ($R^2=0.71$), a single-frame actor is structurally blind on velocity-critical tasks. (ii) *Terminal coverage*: only $\sim 4\%$ of start latents lie within 5 steps of a terminal, so the continue head almost never fires during dreams. (iii) *Latent drift*: open-loop predictor error reaches $\sim 31\%$ relative L_2 at $h=15$, well into the regime where the reward signal is unreliable.

Robust Actor-Critic (AC-v2) We address each failure mode with additive changes; the backbone remains frozen.

- **Temporal context.** Actor and critic consume a flattened history of the last $H=3$ latents, making velocity-like quantities linearly accessible.
- **Distributional pessimistic critic.** A 5-member ensemble of distributional critics with 101 symlog-spaced bins over $[-15, 15]$, cross-entropy against two-hot symlog targets Hafner et al. [2025]; per-state value is the *minimum* over members.
- **Behavior-cloning anchor.** The actor is BC-warm-started for 3000 updates, then refined under a KL-to-BC penalty (coefficient 1.0).
- **Prioritized terminal starts.** Latents within $w=8$ steps of a terminal are weighted $20\times$ during sampling, combined with percentile-based return normalization Hafner et al. [2025].

Results Figure 17 shows eval return over 20k updates on MountainCar for the AC-v1 baseline, an ablation adding only temporal context + KL-to-BC, and full AC-v2. The baseline stays at the floor; both robust variants escape it from the first eval and converge near the behavior-cloning reference. Table 4 reports final eval returns. All AC-v2 runs use seed 42 with shared hyperparameters: $H=3$, 3000 BC pretrain updates, KL coefficient 1.0, ensemble size 5, imagination horizon 5, terminal window 8 with weight 20, entropy scale 10^{-2} .

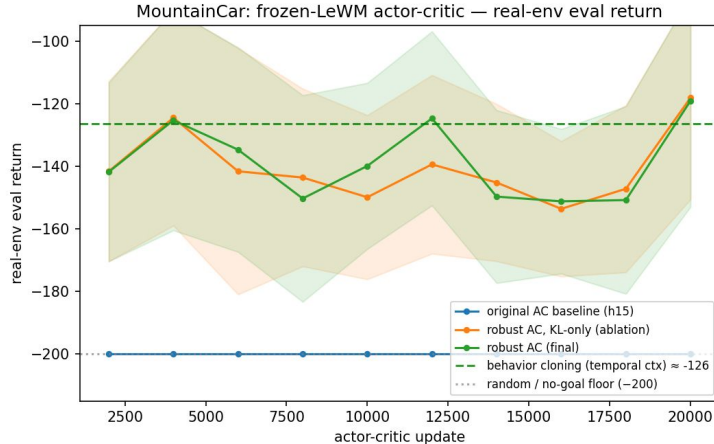


Figure 17: MountainCar real-environment eval return over 20k AC updates (LeWM seed 7). The single-frame baseline stays at the no-goal floor of -200 ; both the KL-only ablation and full AC-v2 escape the floor by the first eval and converge near the BC reference (≈ -126).

Table 4: Real-environment eval return after 20k AC updates on a frozen LeWM (seed 7) backbone. BC reference is behavior cloning from temporal-context latents.

Method	MountainCar	CartPole	Hopper
AC-v1 (single-frame, MSE critic)	-200.0	1.0	2.65
BC reference (temporal context)	-126.5	2.60 ± 1.96	27.95 ± 15.05
AC-v2 (this work)	-119.1	4.54 ± 7.03	22.99 ± 7.04

On MountainCar, AC-v2 recovers from a fully broken baseline to a return statistically indistinguishable from the data-collection policy, despite the latter having hand-coded access to ground-truth velocity. CartPole and Hopper show the same qualitative pattern, AC-v2 lifts the policy well above its v1 floor (CartPole $1.0 \rightarrow 4.54$, Hopper $2.65 \rightarrow 22.99$) and into the same range as the BC reference (CartPole 2.60 , Hopper 27.95). On Hopper the KL anchor pulls AC-v2 down slightly relative to BC alone, consistent with the fact that imagined returns and real returns track closely throughout training (Section above): there is little residual signal left for AC refinement to extract on top of the BC warm-start, and the small remaining drift moves the policy slightly off the BC optimum. CartPole is the one environment where AC refinement clearly improves on BC, plausibly because its denser reward gradient is more informative for value learning than Hopper’s fall-floor or MountainCar’s sparse goal signal.

Limitations AC-v2 matches but does not exceed the data policy on MountainCar, the reward is too flat for further improvement from the imagined return. The recipe also requires a working continuation head; without one, terminal-start prioritization has nothing to weight against. Generalization to settings where the data policy is itself weak remains untested.

E Probe mistake visualisation

This section describes how the probe-mistake plots are produced. The goal of the visualisation is to show *where* in the observation space a probe makes large errors, rather than only reporting an aggregate score.

E.1 Encoder failure modes: LunarLander

Different encoders fail in different regions of the state space. Figure 18 visualises probe-mistake density on LunarLander (return target, no action conditioning) for DINOv2, DreamerV3, and LeWM. LeWM shows pronounced errors in the top-left and top-right corner regions, a pattern not present to the same degree in either DINOv2 or DreamerV3. The maximum error intensity in the LeWM plot

is also substantially higher than in the other two, indicating that LeWM mistakes are not just more localised but also larger in magnitude.

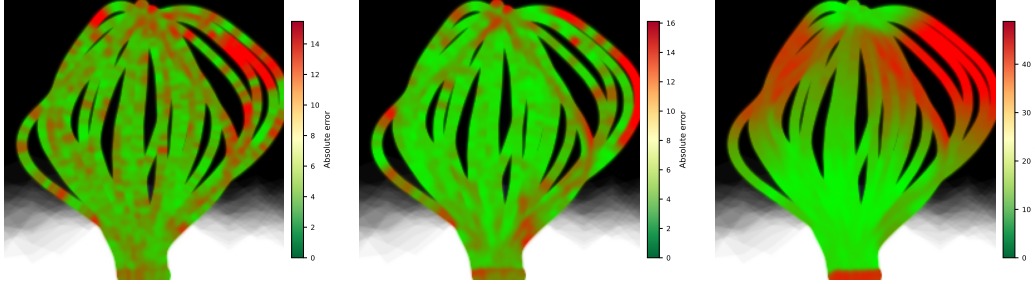


Figure 18: Probe-mistake density on LunarLander for DINOv2 (left), DreamerV3 (middle), and LeWM (right). LeWM shows a clear concentration of error in the upper corners and a higher peak error magnitude than the other two encoders.

E.2 Encoder failure modes: Pong

Figure 19 shows probe-mistake locations on Pong (reward target, no action conditioning). DINOv2 shows no systematic top-bottom asymmetry. DreamerV3 errors are highly localised at the top of the playfield, along the border between the playfield and the white buffer strip above it. LeWM errors are spread more broadly across the upper portion of the frame rather than confined to a single spot. We hypothesise that the scoreboard at the top of the Pong frame interferes with the encoding for the smaller DreamerV3 and LeWM models, which have less capacity and must compress more aggressively; the difference in spatial spread between DreamerV3 (localised) and LeWM (diffuse) suggests the two encoders absorb that interference in qualitatively different ways. DINOv2, as a larger encoder, has the capacity to preserve scene structure separately from the scoreboard region and is less affected.

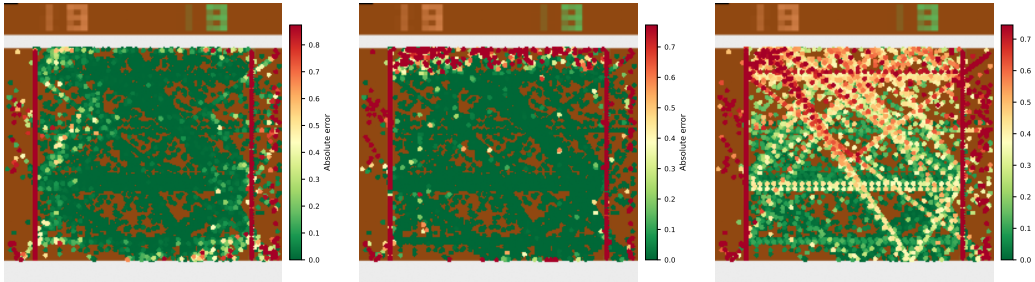


Figure 19: Probe-mistake locations on Pong for DINOv2 (left), DreamerV3 (middle), and LeWM (right). LeWM errors concentrate in the top third of the frame; DreamerV3 shows a weaker version of the same pattern; DINOv2 shows no clear top-bottom asymmetry.

E.3 Per-sample error

For each test sample i , we compute an absolute error e_i between the probe prediction and the ground-truth target.

- **Regression and binary targets.** $e_i = |\hat{y}_i - y_i|$, where $\hat{y}_i$ is the probe output and y_i is the ground-truth reward or discounted return.
- **Multiclass targets.** $e_i = 1 - p_i(y_i)$, the complement of the probability the probe assigns to the true class.

All probes shown in these figures use the linear head (head = linear) trained at horizon $h = 0$.

454 E.4 Tracked object positions

455 Each test sample is associated with one or more image-space positions of a salient object in the
456 corresponding observation. The exact set of objects depends on the environment:

- 457 • **LunarLander.** The lander position, computed analytically from the underlying state vector
458 and rescaled to the 224×224 image grid.
- 459 • **MountainCar.** The car position, computed from the cart x -coordinate using the Gym render
460 geometry (screen 600×400 , world width 1.8, with the cosine track profile).
- 461 • **CartPole.** The pole tip, computed from the cart position and the pole angle, mapped through
462 the Gym render geometry.
- 463 • **Pong.** Three positions per frame, ball, left paddle, right paddle, extracted directly from
464 the raw 210×160 Atari frames by colour thresholding inside the playfield region (rows
465 34–194) and rescaled to the 224×224 image grid.

466 Samples for which a position cannot be recovered (e.g. the ball is missing from a Pong frame) are
467 excluded from that object’s overlay but still contribute to other objects.

468 E.5 Long-exposure background

469 To give the overlay a meaningful visual context, we build a single background image per environment
470 by averaging all test frames in $[0, 1]$ pixel space. This produces a long-exposure-style composite
471 showing where dynamic elements appear most often (e.g. the Pong playfield, the LunarLander pad)
472 while keeping static elements sharp.

473 E.6 Error normalisation

474 Within a group of plots that should be visually comparable (e.g. the three encoders on the same envi-
475 ronment and target), we cap the colour scale at a shared quantile of the per-sample errors. By default
476 this cap is the 95th percentile, $\tau = P_{95}(\{e_i\})$, computed either per-run or as the maximum/minimum
477 over the group of runs being compared. Each error is normalised as

$$\tilde{e}_i = \min\left(\frac{e_i}{\tau}, 1\right).$$

478 Samples with $e_i > \tau$ are flagged as out-of-range and rendered in a distinct dark-red colour, so that
479 very large errors remain visible without saturating the rest of the scale. The cap value used for each
480 figure is reported in its automatically generated title.

481 E.7 Rendering modes

482 We use two complementary rendering modes.

483 **Dots.** A small disc of radius r pixels is drawn at each tracked object position, coloured by $\tilde{e}_i$
484 through the reversed Red–Yellow–Green colormap (green = low error, red = high error, dark red =
485 out-of-range). Discs are drawn in order of error magnitude so that high-error dots appear on top of
486 low-error dots (or the inverse, depending on the variant). The resulting overlay is alpha-blended onto
487 the long-exposure background.

488 **Density.** Instead of discrete dots, each sample deposits two small Gaussian blobs centred at its
489 object position(s), one weighted by $1 - \tilde{e}_i$ (correct) and one weighted by $\tilde{e}_i$ (wrong). Out-of-range
490 samples deposit into a separate dark-red weight field. At every pixel the three weight fields are
491 normalised to ratios, which determine the displayed colour. The overlay alpha saturates as soon as
492 any single sample has been deposited, so a region visited once and a region visited a hundred times
493 appear equally vivid, the colour encodes the *ratio* of correct to wrong samples, not the visit count.
494 This is what produces the smooth colour gradients in the LunarLander figures.

495 **E.8 Reading the plots**

- 496 • Bright green regions indicate that, whenever the tracked object is in that region, the probe
497 predicts the target accurately.
- 498 • Red and dark-red regions mark systematic spatial failure modes: states where the encoded
499 (or predicted) latent does not retain enough information to recover the target.
- 500 • Comparing the same plot across encoders, with a shared colour cap, highlights which parts
501 of the state space each encoder finds difficult.

502 This method is intentionally agnostic to the absolute scale of the error: by normalising to a within-
503 group quantile and using a shared colour scale, the figures expose spatial structure in the errors rather
504 than which encoder happens to have the largest mean error.

NeurIPS Paper Checklist

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper’s contributions and scope?

Answer: [Yes]

Justification: The abstract and introduction clearly state the paper’s scope and contributions. The paper investigates whether JEPA-based latent world model representations encode recoverable reward and value information useful for downstream control through probing, finetuning, and reward-guided MPC evaluation. Claims are limited to the evaluated environments and experimental settings.

Guidelines:

- The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
- The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
- The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
- It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [Yes]

Justification: The paper discusses several limitations, including strong environment dependence, degradation at long rollout horizons, and the gap between reward recoverability and controllability. The paper also acknowledges that latent rollout inaccuracies remain a major bottleneck for downstream planning performance.

Guidelines:

- The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate “Limitations” section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren’t acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [N/A]

Justification: The paper does not present theoretical results, proofs, or formal guarantees.

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: The paper provides detailed descriptions of environments, probing setups, horizons, window sweeps, probe architectures, training procedures, evaluation metrics, and data splits in the appendix.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general, releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
 - (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
 - (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
 - (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
 - (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility.

In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [Yes]

Justification: Code, configurations, and instructions for reproducing experiments will be released anonymously in the supplemental material.

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. Experimental setting/details

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [Yes]

Justification: The paper specifies training details, evaluation metrics, probe architectures, optimization settings, horizons, temporal windows, seeds, and dataset splits necessary to reproduce the experiments.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. Experiment statistical significance

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [Yes]

Justification: Figures report mean and standard deviation across probe seeds and encoder seeds. The appendix explicitly states the seeds used and describes the reported statistics.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.

- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. Experiments compute resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [Yes]

Justification: The appendix specifies the hardware used for experiments, including GPU types, approximate runtime requirements, and the relative computational cost of probing, finetuning, and MPC evaluation experiments. Compute requirements for large hyperparameter sweeps and finetuning runs are also discussed.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. Code of ethics

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines>?

Answer: [Yes]

Justification: The research complies with the NeurIPS Code of Ethics. Experiments are conducted on standard reinforcement learning benchmarks and do not involve human subjects or sensitive data.

Guidelines:

- The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. Broader impacts

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [No]

Justification: This work is primarily foundational research on latent world models and downstream control. The paper does not explicitly discuss broader societal impacts.

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. Safeguards

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [N/A]

Justification: The paper does not release high-risk generative models, scraped datasets, or systems with significant misuse potential.

Guidelines:

- The answer [N/A] means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. Licenses for existing assets

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [Yes]

Justification: The paper cites all external models, environments, and prior methods used, including Gymnasium, DINOv2, DreamerV3, and LeWM.

Guidelines:

- The answer [N/A] means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, paperswithcode.com/datasets has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. New assets

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [Yes]

Justification: Released code and experiment configurations will be documented with instructions for reproducing the experiments.

Guidelines:

- The answer [N/A] means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. Crowdsourcing and research with human subjects

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [N/A]

Justification: The paper does not involve crowdsourcing or human subjects.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. Institutional review board (IRB) approvals or equivalent for research with human subjects

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

820 Answer: [N/A]
821 Justification: The paper does not involve human subjects research.
822 Guidelines:
823 • The answer [N/A] means that the paper does not involve crowdsourcing nor research
824 with human subjects.
825 • Depending on the country in which research is conducted, IRB approval (or equivalent)
826 may be required for any human subjects research. If you obtained IRB approval, you
827 should clearly state this in the paper.
828 • We recognize that the procedures for this may vary significantly between institutions
829 and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the
830 guidelines for their institution.
831 • For initial submissions, do not include any information that would break anonymity (if
832 applicable), such as the institution conducting the review.
833 **16. Declaration of LLM usage**
834 Question: Does the paper describe the usage of LLMs if it is an important, original, or
835 non-standard component of the core methods in this research? Note that if the LLM is used
836 only for writing, editing, or formatting purposes and does *not* impact the core methodology,
837 scientific rigor, or originality of the research, declaration is not required.
838 Answer: [N/A]
839 Justification: LLMs are not part of the proposed methods or experiments.
840 Guidelines:
841 • The answer [N/A] means that the core method development in this research does not
842 involve LLMs as any important, original, or non-standard components.
843 • Please refer to our LLM policy in the NeurIPS handbook for what should or should not
844 be described.